

# ECE4132 assignment 3: PI control of the temperature control lab system

Prepared by: Dr. James Saunderson

---

## Aims

The aim of this assignment is to gain familiarity with analysing the effects of PI controller parameters using both the root locus plot (in MATLAB) and simulations (using Simulink).

## References and resources

- ‘Getting started with Simulink’ videos from Mathworks: <https://au.mathworks.com/videos/series/getting-started-with-simulink.html>
- K. J. Åström and R. M. Murray *Feedback systems: an introduction for scientists and engineers* Princeton University Press

## 1 Approximating the TClab system with a second-order model

In assignment 2 we found that the TClab system was well approximated by a first-order model with time delay. Recall that a linear time-invariant system follows a first-order with time delay (FOTD) model if it has transfer function

$$\frac{Ke^{-\tau s}}{s + a}$$

where  $\tau$  and  $a$  and  $K$  are constants. Since the delay does not have a rational transfer function, it is common to approximate the delay via the first-order approximation

$$e^{-\tau s} \approx \frac{1 - s\tau/2}{1 + s\tau/2}.$$

To see why this is a good approximation, observe that the Taylor expansions of both sides are

$$\begin{aligned} e^{-\tau s} &= 1 - \tau s + \tau^2 s^2/2 - \tau^3 s^3/6 + \dots \\ \frac{1 - s\tau/2}{1 + s\tau/2} &= 1 - \tau s + \tau^2 s^2/2 - \tau^3 s^3/4 + \dots \end{aligned}$$

which agree up to the second-order terms. (This is an example of a rational function approximation called a Padé approximation.)

**Second order system model:** Using this approximation of a delay, we can approximate a first-order model with time delay using a second order model of the form

$$P(s) = \frac{(1 - s\tau/2)K}{(1 + s\tau/2)(s + a)}. \quad (1)$$

In the case of the temperature control system, let  $y(t)$  denote the temperature at time  $t$ , let  $y_{\text{room}}$  denote the room temperature, and let  $u(t)$  denote the heater input at time  $t$ . Throughout the assignment we will model the temperature control system heater input  $u(t)$  and output  $D(t) = y(t) - y_{\text{room}}$  by a second-order system of the form  $P(s)$ .

**Parameters for second order system model:** Throughout the assignment, we will use the parameter values

- $a = 0.0102$
- $K = 0.0051$
- $\tau = 11$
- $y_{\text{room}} = 23$ .

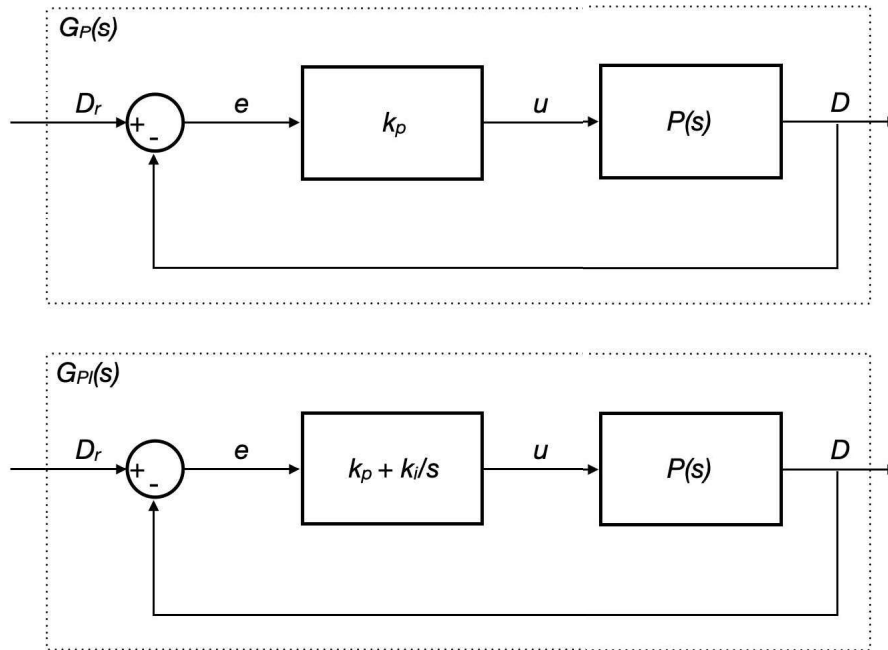
in the second-order system model whenever numerical answers are required. These are similar to the parameters we found in assignment 2 that gave a good approximation of the temperature control system.

## 2 Analysis of P and PI control of the second order model

In this section you will analyse the closed loop system consisting of the model  $P(s)$  (defined in (1)) connected in feedback with either

- a proportional controller with transfer function  $C(s) = k_p$  or
- a PI controller with transfer function  $C(s) = k_p + \frac{k_i}{s}$ .

A block diagram of these systems is shown below.



Note that in this diagram, the system output is  $D(t) = y(t) - y_{\text{room}}$  (the temperature deviation), and the system input is  $D_r(t) = r(t) - y_{\text{room}}$ , the reference temperature minus the room temperature.

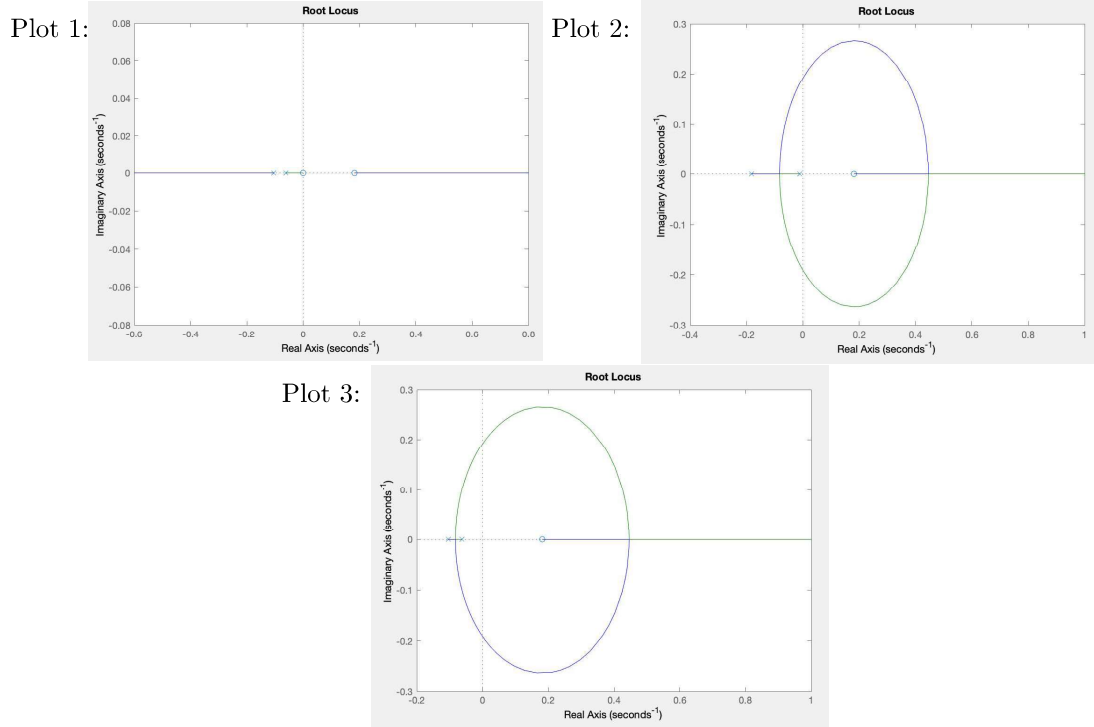
### 2.1 Analysis of proportional control

Answer the following questions.

- Q1 Find an expression for the transfer function  $G_P(s)$  from  $D_r$  to  $D$  when  $C(s) = k_p$ . Express your answer in terms of  $a, K, \tau, k_p$  and  $s$ .
- Q2 Assuming the closed-loop system is stable, write down an expression for the steady-state value of the error  $D_r - D$  when  $C(s) = k_p$  and the reference is  $D_r(t) = 1(t)$ , the unit step. Express your answer in terms of  $a, K, \tau$  and  $k_p$ .

In a MATLAB script, write code to define the system parameters, and set up an appropriate transfer function object for  $P(s)$ . Add code to plot the root locus that shows the locations of the poles of  $G_P(s)$  as  $k_p$  varies from 0 to  $\infty$ .

Q3 Which of the following plots shows the root locus plot of the poles of  $G_P(s)$  as  $k_p$  goes from 0 to  $\infty$ ? (Use the parameter values in Section 1).



Q4 Correctly complete the statement regarding the root locus plot from Q3.

Because the (plant/controller) has a (pole in the RHP/pole in the LHP/zero in the RHP/zero in the LHP) at least one branch of the root locus (ends in the RHP/ends in the LHP/starts in the RHP/ends in the RHP). Therefore, for large enough values of  $k_p$  the (closed loop system is BIBO stable/closed loop system is not BIBO stable/open loop system is BIBO stable/open loop system is not BIBO stable).

Next we consider the trade-off between stability of the closed loop system and steady-state reference tracking error. For these questions, please give numerical answers. You may want to first find symbolic expressions (in terms of all of the relevant problem parameters) which you then evaluate numerically.

Q5 Find the smallest positive value of  $k_p$  for which  $G_P(s)$  is not BIBO stable. Give a numerical answer correct to two significant figures.

Q6 Consider connecting  $P(s)$  in feedback with a proportional controller so that the resulting closed-loop system is stable. If the input is  $D_{\text{ref}} = 60 - 23 = 37$ , find the smallest possible steady-state error that could be achieved. Give a numerical answer correct to two significant figures.

Your answers to Q5 and Q6 should highlight that using proportional control for this system has its limitations. In particular, we cannot make the steady-state error as small as we want while maintaining stability.

## 2.2 Analysis of PI control

To achieve a smaller steady-state reference tracking error than your answer to Q6, we will use a PI controller.

Q7 Find an expression for the transfer function  $G_{PI}(s)$  from  $D_r$  to  $D$  when  $C(d) = k_p + k_i/s$ . Express your answer in terms of  $a, K, \tau, k_p, k_i$  and  $s$ .

Q8 Assuming the closed-loop system is stable, write down an expression for the steady-state value of the error  $D_r - D$  when  $C(s) = k_p + k_i/s$ ,  $k_i \neq 0$ , and the reference is  $D_r(t) = \mathbf{1}(t)$ . Express your answer in terms of  $a, K, \tau, k_p$  and  $k_i$ .

Now that we understand the steady-state error when using a PI controller, let's turn our attention to understanding how the parameters  $k_p$  and  $k_i$  affect the locations of the closed loop poles. We will do this by fixing some values of  $k_p$  and using the `rl locus` command to plot the poles of  $G_{PI}(s)$  as  $k_i$  ranges from 0 to  $\infty$ .

(This can be done using a variation on the method introduced at the end of workshop 4. In the workshop, the `rl locus` command was used to plot the poles of a closed-loop system obtained by connecting a PD controller in feedback with a plant, as the parameter  $k_d$  varied from 0 to  $\infty$ , for a fixed value of  $k_p$ . You may wish to review this to help answer the following questions.)

Q9 For a given value of  $k_p$ , the poles of  $G_{PI}(s)$  are the solutions (for  $s$ ) of the equation

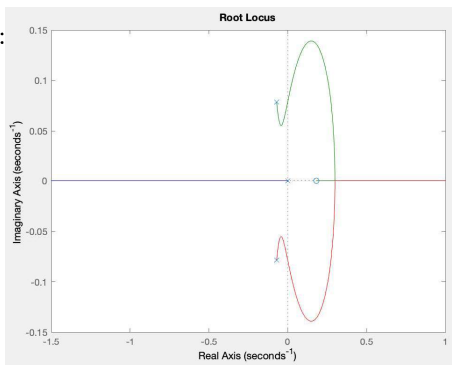
$$A(s) + k_i B(s) = 0$$

for suitable polynomials  $A(s)$  and  $B(s)$ . Find expressions for  $A(s)$  and  $B(s)$ . Express your answers in terms of  $K, a, \tau, k_p$  and  $s$ .

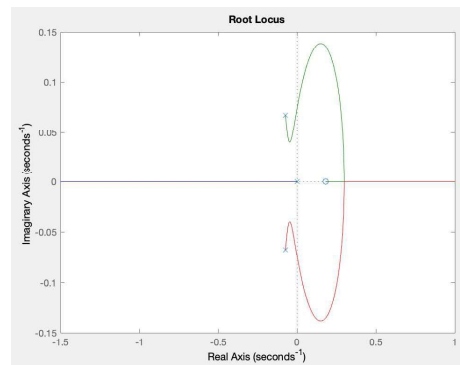
For each of  $k_p = 5$ ,  $k_p = 6.25$ ,  $k_p = 7.5$ , and  $k_p = 10$ , use the `rl locus` command to plot the poles of  $G_{PI}(s)$  as  $k_i$  ranges from 0 to  $\infty$ . You should produce four plots.

Q10 Match the values of  $k_p$  to the root locus plots of the poles of  $G_{PI}(s)$  as  $k_i$  ranges from 0 to  $\infty$ .

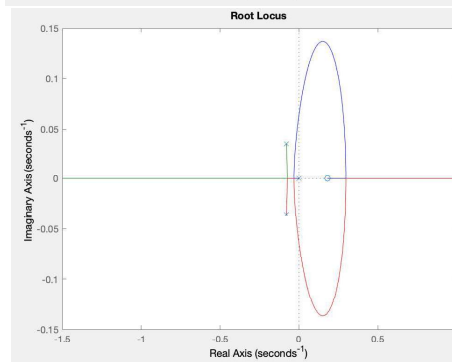
Plot 1:



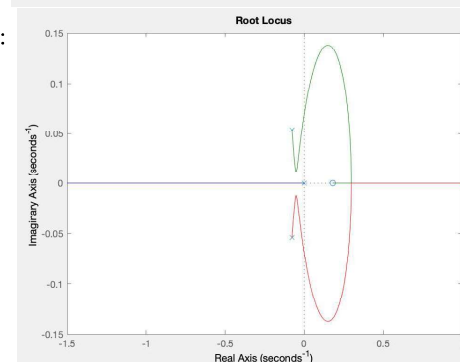
Plot 2:



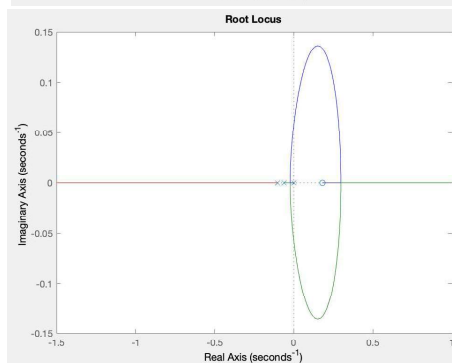
Plot 3:



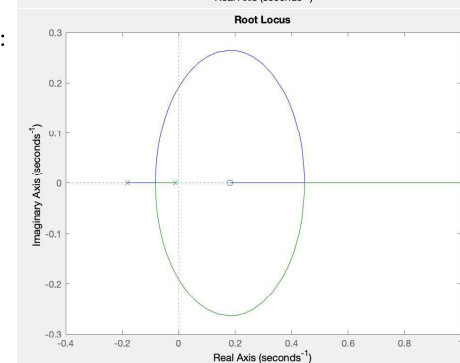
Plot 4:



Plot 5:



Plot 6:



Which plot matches to:

- $k_p = 5$ ?
- $k_p = 6.25$ ?
- $k_p = 7.5$ ?
- $k_p = 10$ ?

Q11 Complete the statement related to the plots of the poles of  $G_{PI}(s)$  for different fixed values of  $k_p$ . In both of the cases ( $k_p = 5$  and  $k_p = 10/k_p = 5$  and  $k_p = 6.25/k_p = 5$  and  $k_p = 7.5/k_p = 6.25$  and  $k_p = 7.5/k_p = 6.25$  and  $k_p = 10$ ) it (is/is not) possible to find a value of  $k_i$  such that the poles of  $G_{PI}(s)$  are all real and the closed loop system is BIBO stable.

### 3 PI control of the second-order system model

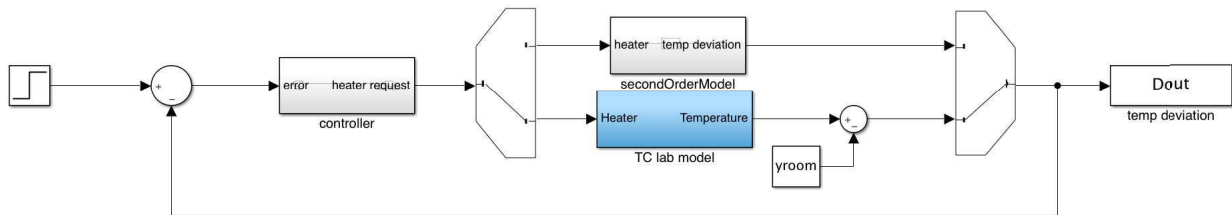
In this section you will add a PI controller to a Simulink model that implements the second-order model of the temperature control system as well as the TClab system itself.

We will aim to achieve the specifications:

- zero steady-state error
- percentage overshoot of at most 5%
- settling time of at most 120 seconds
- rise time of at most 50 seconds

However, our focus is less on tuning the parameters to achieve this, and more on understanding

The Simulink model you will add a controller to is in the file `TClabControl.slx`.



This Simulink model has a two-way-switch that allows you to simulate either

- (both switches in the upward position) the second-order model of the plant
- (both switches in the downward position) the actual TClab system

To change the position of the switches, double-click on the switch block. Also double click on the second-order model subsystem block to check that you understand how it is implemented, and which variable names you need to define to specify the parameters of the model.

- Inside the controller subsystem, implement a PI controller (using ‘gain’ blocks, an ‘integrator’ block, ‘sum’ blocks, and a constant block). For the values of the gains  $k_i$  and  $k_p$ , just use variable names **ki** and **kp**.
- Create a MATLAB script called **setupModelParameters**. Use it to set the values of the variables **P** and **yroom**. Use it to set the value of **ref** so that the system temperature will be regulated at 60 degrees.
- Initially, set the values of  $k_p = 7.5$  and set  $k_i = 0.153$ . Note that for  $k_p = 7.5$ , this is the value of  $k_i$  that gives complex conjugate closed loop poles with the smallest damping ratio ( $\zeta \approx 0.97$ , to two significant figures).
- Make sure the variant source and sink are routing the signal through the second-order model.

- Simulate the system for 600 seconds.
- Use the MATLAB `stepinfo` function to compute step response parameters of the system output (Dout) in your MATLAB workspace.

Q12 The percentage overshoot given by `stepinfo` is \_\_\_\_\_. (Answer correct to two significant figures.)

This value is (different than/the same as) the result of the formula  $100\% \times e^{-\pi\zeta/\sqrt{1-\zeta^2}}$  because this (is/is not) a canonical second order system.

- *Some things to try and to think about:* Try choosing different values of  $k_i$  and  $k_p$  corresponding to points on the root locus plots you found in Q10. Simulate the closed-loop system and plot the response to see what it looks like. If you choose gains that put the poles close to the imaginary axis, you should see that the root locus plots predict the onset of instability well. However, you should also see that the damping ratio of any complex-conjugate poles is not a good predictor of the percentage overshoot. This is because the zeros of the closed loop system (at  $-k_i/k_i$  and at  $2/\tau$ ) are quite close to the imaginary axis, and so have a fairly strong effect on the system response.
- *Optional:* try to find parameters  $k_i$  and  $k_p$  that achieve the specifications given above. (Just below we give an example of such a choice.)

A set of parameters that achieves the specifications is  $k_p = 8.75$  and  $k_i = 0.1$ .

- Using these parameters, simulate the closed loop system. Plot the system response. Use the `stepinfo` command to find the percentage overshoot, rise time, and settling time of the system response.

Q13 What are the percentage overshoot, rise time, and settling time of the system response? Give each of your answers to two significant figures.

### 3.1 Evaluating on the TClab system

- Double click the ‘variant source’ and ‘variant sink’ blocks in the Simulink model to switch to the TClab model.
- Simulate the closed-loop system with the final values of  $k_i = 0.1$  and  $k_p = 8.75$ . Plot the system response. Use the `stepinfo` function to find the percentage overshoot and the rise time.

Q14 What are the percentage overshoot and rise time of the closed-loop system response using the TClab plant? Give each of your answers to two significant figures.

There is a major discrepancy between the response of our model of the system, and the actual response of the system. The real system shows a longer rise time, a larger overshoot, and a longer settling time! This is due to *actuator saturation* in the TClab system (the heater input cannot exceed 100). We will investigate this further in Assignment 4, including ways to mitigate some of the effects of actuator saturation.